TEAM 5 | Tech Titan

Brick Oracle

James, Owen, Theodore, Woobin

Problem and Motivation

As a LEGO collector with loose bricks from bins and deconstructed sets how can I identify what sets can build from my free bricks.

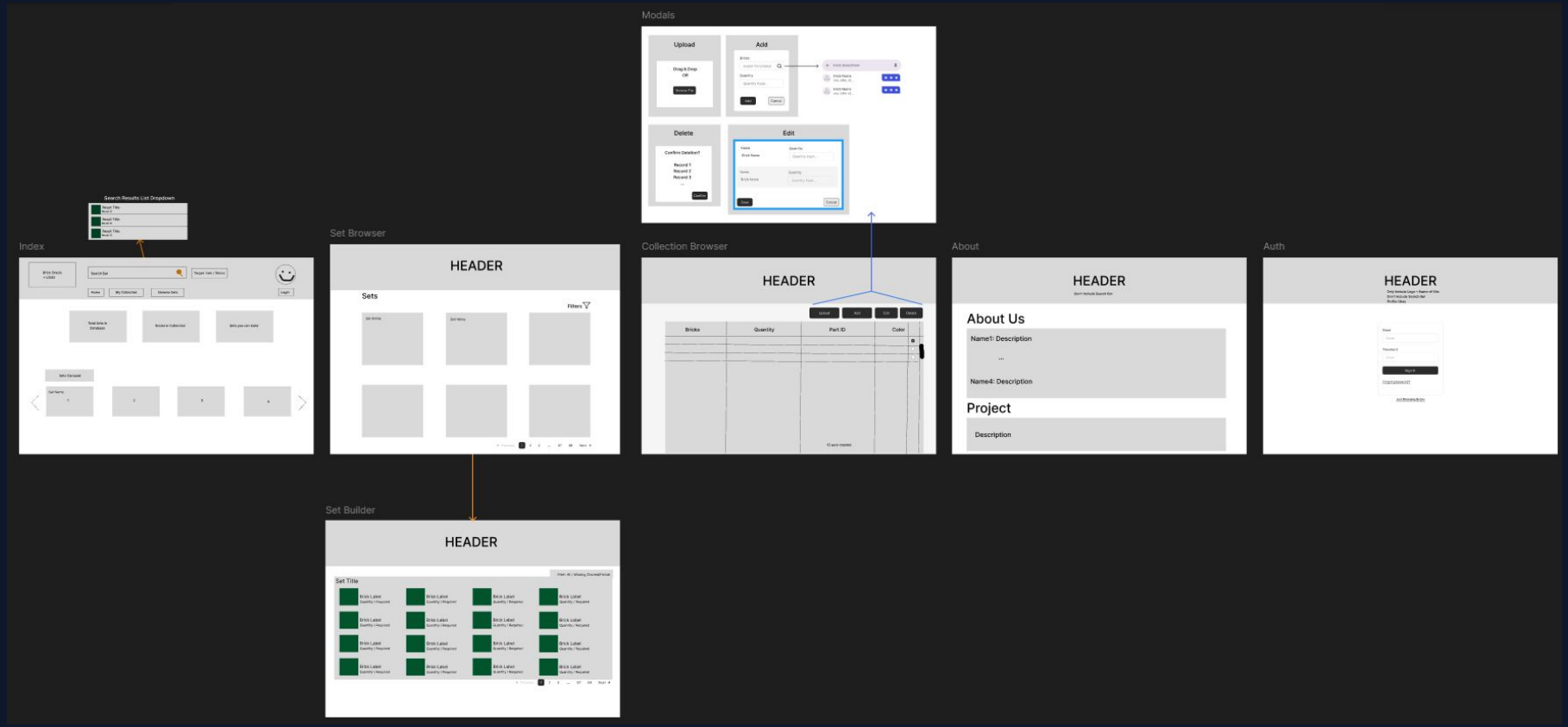
With thousands of loose bricks, keeping track of potential builds manually is nearly impossible, leaving collections gathering dust, rather than displayed on your shelf.

Solution

Brick Oracle lets you maximize your collection instantly.

- Upload your loose brick collection
- Browse the comprehensive catalog
- See exactly what parts you are missing

Architecture



Tech Stack

Frontend

React + TypeScript

Build interactive user interfaces with a component-based architecture and type safety.

Backend

Flask (Python)

Build flexible and fast servers and control business logic using a lightweight API framework.

Database

SQLite

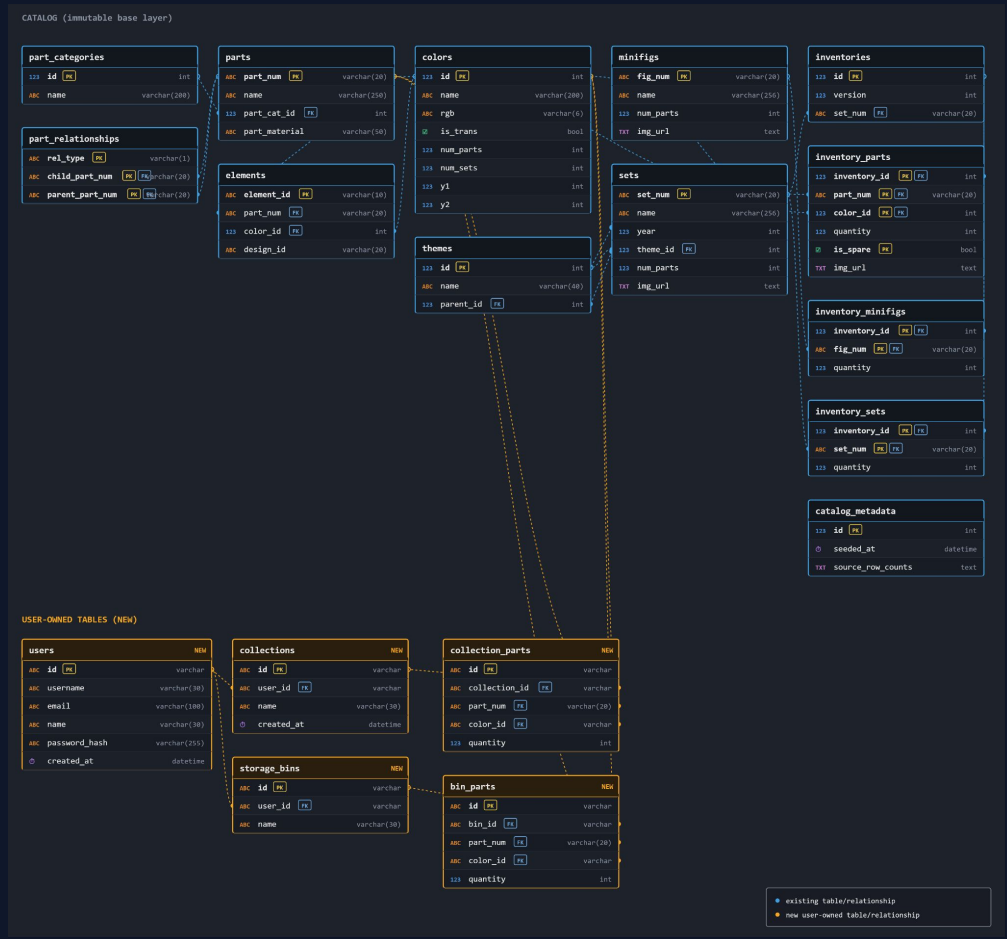
Seeded with 1.5M rows of Rebrickable data.

- 17 tables configured
- 12 catalogs
- 5 user-owned tables

Schema

Mimicked Rebrickable Schema for catalog seeding and records

Added 5 new tables: Users, Collections, Collection Parts, Storage Bins, Bin Parts



Key Features

1. Home page & Set Carousel
2. Login + Authentication Flow
3. User Collection Viewer
4. Set Browser with filters
5. Set Bricks Viewer
6. Brick Diff of user collection and selected Set

Demo Overview

Our live demo will follow this sequence:

1. View Home Page with set carousel
2. Create a new user
3. Login and view the user’s collection
4. Navigate to the Set Browser
5. Browse the available sets
6. Filter the sets to narrow our interests
7. Select a set to view its contents

Demo

End - Thank you!

Brick Oracle GitHub

Developers:

- Woobin Huh
- James Tillman
- Theodore Matthews
- Owen Ahlers

What went well

- Focused task completion and timely execution with full participation
- Distributed workflow and project management for an asynchronous group using agreed upon tooling as intended
- Timely feedback on progress made, and flexibility to suggested changes following established conventions for consistent execution

Challenges

Data Seeding

Seeding 1.5M rows without killing application startup time.

Optimizing initial database setup and migration scripts to handle Rebrickable datasets efficiently during application launch.

CSS Styling

CSS naming collisions across branches.

Managing styles in a collaborative team environment and preventing global namespace pollution across feature branches.

Data Diff

Building the diff UI and API against a shared data model.

Aligning the React frontend and Flask backend to process, calculate, and display real-time inventory differences accurately.

Future Work

- Mobile first redesign for easy scanning and uploading bricks
- 3D visualization for building LEGO with existing collection
- Data sync of newly added sets from the LEGO catalog
- Switch from SQLite to Postgres to allow for multiple users in a production environment